PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Taller investigativo Arquitecturas de Software.

Presentado a: **Yoly Alexandra Muñoz**

Por Aprendiz: **Cristian David Montilla Ordoñez**

Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 9 de octubre del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

3. Fase I. Investigación técnica

1. Arquitectura Monolitica.

- **Definición:** La arquitectura monolítica es una forma de crear un programa donde casi todas sus funciones están dentro de una misma aplicación. Por ejemplo una pagina web podemos tener registro de usuarios, los cursos, las inscripciones y las evaluaciones dentro de un mismo proyecto.
- **Problema a resolver:** Busca que sea mas sencillo desarrollar y poner en funcionamiento una aplicación, sin tener que dividirla desde el principio en muchos programas diferentes. Esto ayuda cuando apenas estamos comenzando un proyecto.
- **Componentes principales y función de cada uno:**

  Interfaz de usuario: es la parte que ve y utiliza la persona, como los botones, formularios y menús.

  Lógica del negocio: contiene las instrucciones para que el programa haga lo que necesitamos. Por ejemplo, comprobar si un estudiante puede inscribirse en un curso.

  Acceso a datos: se encarga de realizar las consultas y guardar información.

  Base de datos: guarda los datos de los usuarios, cursos, calificaciones y demás información.

- **Forma en que se comunican los componentes:**

  Las partes se comunican dentro de la misma aplicación. Por ejemplo, cuando un estudiante se inscribe en un curso, el programa revisa los datos, verifica que pueda inscribirse y guarda la información en la base de datos.

- **Ventajas y Desventajas.**

Ventajas	Desventajas
Es más fácil comenzar a desarrollar porque no necesitamos configurar muchos servicios diferentes. Esto puede ayudar a terminar una primera versión del proyecto más rápido.	Cuando el programa crece mucho, puede volverse difícil encontrar y organizar el código.
Puede ser más sencillo encontrar errores porque las funciones están dentro del mismo proyecto.	Si necesitamos mejorar una parte que consume muchos recursos, posiblemente tengamos que aumentar la capacidad de toda la aplicación, aunque las otras funciones no lo necesiten.

- **Ejemplo real o técnicamente verificable:**

Según consultas una página web hecha con Laravel o Django que tenga el registro de usuarios, el catálogo de productos y las compras dentro de una misma aplicación.

- **Un proyecto de software en la que seria apropiada:**

La utilizaría para crear una primera versión de mi proyecto PlayNoteMusic, donde los estudiantes pueden registrarse, consultar cursos musicales, inscribirse y realizar evaluaciones.

- **Una situación en la que no recomendaría utilizarla y la justificación:**

No la recomendaría para un sistema demasiado grande, en el que diferentes partes necesiten funcionar y actualizarse por separado. En

ese caso, tener todo dentro de una sola aplicación podría hacer más difíciles los cambios.

- **Una fuente técnica y el dato especifico que tuvo de ella:**

Martin Fowler: MonolithFirst

La fuente explica que empezar con una aplicación monolítica puede ser una buena decisión, porque al principio todavía estamos conociendo las necesidades del proyecto y no siempre es necesario dividirlos en varios servicios.

2. Arquitectura en capas (N-tier)

- **Definición:** Esta arquitectura consiste en organizar el programa por partes, donde cada una tiene una tarea diferente. Por ejemplo, una parte muestra la página al usuario, otra se encarga de las reglas del programa y otra trabaja con la base de datos.
- **Problema a resolver:** Busca evitar que todo el código esté mezclado en un solo lugar. De esta manera, es más fácil encontrar dónde se hace cada cosa y realizar cambios sin afectar innecesariamente otras partes.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

- **Componentes principales y funciones de cada uno:**

  Capa de presentación: muestra la información y permite que el usuario utilice el sistema.

  Capa de negocio: contiene las reglas que debe cumplir el programa.

  Capa de acceso a datos: realiza las operaciones necesarias para consultar o guardar información.

  Base de datos: conserva la información para poder utilizarla después.

- **Forma en que se comunican los componentes:**

Normalmente, una capa le pasa la información a otra. Por ejemplo, cuando un estudiante llena el formulario de inscripción, la pantalla envía los datos, el programa verifica que estén correctos y después se guardan en la base de datos.

- Ventajas y Desventajas

Ventajas	**Desventajas**
El código queda mejor organizado. Si necesitamos cambiar una regla de inscripción, podemos buscarla en la parte encargada de la lógica del programa.	crear demasiadas capas puede hacer que el proyecto tenga muchos archivos y que tareas sencillas terminen necesitando varios pasos.
facilita el trabajo en equipo, porque cada persona puede encargarse de una parte diferente.	si la información debe pasar por muchas capas, el proceso puede volverse más complicado de seguir.

- **Un ejemplo real o técnicamente verificable de uso.**

Según consultas una aplicación desarrollada con ASP.NET Core que tenga una parte para mostrar la información, otra para procesar las solicitudes y otra para consultar los datos guardados.

- **Un proyecto de software en el que sería apropiado:**

La utilizaría en mi aplicación de PlayNoteMusic para separar las pantallas, las reglas de inscripción a los cursos y las consultas a la base de datos MySQL.

- **Una situación en la que no recomendaría utilizarla y la justificación:**

No utilizaría muchas capas para un programa muy pequeño que solamente tenga una función sencilla. En ese caso, organizarlo de una manera demasiado compleja podría hacer que el desarrollo fuera más difícil de lo necesario.

- **Una fuente técnica consultada y el dato especifico que tuvo ella:**

Microsoft Learn: Architect modern web applications with ASP.NET Core and Azure

La documentación muestra como organizar una aplicación en partes con responsabilidades diferentes, esto ayuda a que el código sea más fácil de mantener y probar.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

3. Arquitectura cliente-servidor

- **Definición:** Es una arquitectura en la que un dispositivo solicita información o algún servicio y otro equipo, llamado servidor, se encarga de responder. Por ejemplo, cuando entramos a una página web, el navegador solicita la información y el servidor se encarga de enviarla.
- **Problema a resolver:** Permite que varios usuarios utilicen una misma aplicación desde diferentes dispositivos, mientras la información y muchas de las operaciones se manejan desde un servidor.
- **Componentes principales y función de cada uno:**

  Cliente: es el dispositivo o programa que utiliza el usuario, como un navegador.

  Servidor: recibe las solicitudes y se encarga de procesarlas.

  Base de datos: guarda la información que necesita la aplicación.

  Red: permite que el cliente y el servidor se comuniquen.

- **Forma en que se comunican los componentes:**

El cliente envía una solicitud al servidor por medio de la red. El servidor procesa la solicitud y devuelve una respuesta. Por ejemplo, cuando un estudiante consulta sus cursos, el servidor busca la información y se la muestra en la página.

- **Ventajas y Desventajas**

Ventajas	Desventajas
la información puede manejarse desde un lugar central. Por ejemplo, cuando se actualiza un curso, los usuarios	si el servidor falla y no existe otro que lo reemplace, los usuarios pueden quedarse sin acceso.

pueden consultar la información actualizada sin tener que modificar sus propios dispositivos.	
varias personas pueden utilizar la misma aplicación desde diferentes computadores o celulares.	si muchas personas utilizan el sistema al mismo tiempo, el servidor puede ponerse lento si no tiene suficiente capacidad.

- **Un ejemplo real o técnicamente verificable de uso:**

Según consultas una página web que funciona desde un navegador, utiliza un servidor web como Nginx y guarda sus datos en MySQL.

- **Un proyecto de software en el que utilizaría:**

La utilizaría para un sistema de citas médicas en el que los pacientes puedan consultar los horarios disponibles y solicitar sus citas desde sus celulares o computadores.

- **Una situación en la que no recomendaría utilizar y la justificación:**

No dependería únicamente de un servidor central para una aplicación que deba seguir funcionando cuando no hay conexión a internet. En ese caso, sería necesario estudiar una solución que permita guardar información y trabajar sin conexión durante cierto tiempo.

- Una fuente técnica consultada y el dato especifico de ella:

Microsoft Learn: Architecture styles

La documentación explica diferentes maneras de organizar un sistema y permite comprender cómo se pueden separar los programas que solicitan servicios de aquellos que se encargan de responder.

4. Arquitectura de microservicios

- **Definición:** Esta arquitectura consiste en dividir una aplicación grande en varios servicios pequeños. Cada servicio se encarga de una función específica. Por ejemplo, uno puede manejar los usuarios, otro los cursos y otro las evaluaciones.
- **Problema a resolver:** Busca que las diferentes partes de una aplicación puedan desarrollarse, actualizarse y ampliarse por separado. Así, un cambio en el servicio de cursos no tendría que obligarnos a actualizar toda la aplicación.
- **Componentes principales y función de cada uno:**

  Microservicios: son los pequeños servicios que realizan las funciones del sistema.

  API Gateway: recibe las solicitudes y las envía al servicio que corresponde.

  Bases de datos: guardan la información que necesita cada servicio, cuando se utiliza una base de datos separada para cada uno.

  Sistema de comunicación: permite que los servicios intercambien información.

  Herramientas de monitoreo: ayudan a revisar si los servicios están funcionando bien o si presentan errores.

- **Comandos principales y función de cada uno**

  Microservicios: son los pequeños servicios que realizan las funciones del sistema.

  API Gateway: recibe las solicitudes y las envía al servicio que corresponde.

Bases de datos: guardan la información que necesita cada servicio, cuando se utiliza una base de datos separada para cada uno.

Sistema de comunicación: permite que los servicios intercambien información.

Herramientas de monitoreo: ayudan a revisar si los servicios están funcionando bien o si presentan errores.

- **Forma en que se comunican los componentes:**

Los servicios pueden comunicarse mediante solicitudes por internet o intercambiando mensajes. Por ejemplo, cuando se realiza una compra, el servicio de pedidos puede enviar un mensaje al servicio de notificaciones para informar que se creó un nuevo pedido.

- **Ventajas y Desventajas**

Ventajas	**Desventajas**
permite mejorar solamente la parte que necesita más capacidad. Por ejemplo, si muchas personas consultan los cursos, podemos aumentar la capacidad del servicio del catálogo sin tener que ampliar todos los demás.	hay más cosas que configurar y revisar, porque ya no tenemos una sola aplicación, sino varios servicios que deben funcionar correctamente.
diferentes equipos pueden trabajar en servicios distintos y actualizarlos por separado, siempre que respeten la forma de comunicarse con los demás.	si un servicio falla, puede afectar las funciones que dependen de él. Por eso, es necesario tener en cuenta los errores y las comunicaciones entre servicios.

- Un ejemplo real o técnicamente verificable de uso:

Una tienda virtual puede organizarse en servicios separados para gestionar los usuarios, los productos, los pedidos y los pagos.

Microsoft explica este tipo de organización en su documentación de microservicios.

- **Un proyecto de software en el que sería apropiado:**

La utilizaría en una plataforma educativa grande, con muchos estudiantes, cursos, evaluaciones y notificaciones que necesiten administrarse y ampliarse por separado.

- **Una situación en la que no recomendaría utilizarla y la justificación:**

No la recomendaría para un proyecto pequeño desarrollado por una sola persona, porque administrar muchos servicios puede tomar más tiempo que desarrollar las funciones del sistema. En ese caso, sería mejor comenzar con una aplicación sencilla y dividirla solamente cuando exista una necesidad.

- **Una fuente técnica consultada y el dato especifico que tuvo ella:**

Microsoft Learn: Microservices architecture style

Esto explica que los microservicios se encargan de funciones específicas y pueden desarrollarse, desplegarse y ampliarse de manera independiente.

5. Arquitectura orientada a servicios (SOA)

- **Definición:** La arquitectura SOA organiza un sistema utilizando servicios que pueden ser aprovechados por diferentes aplicaciones. Por ejemplo, una empresa puede tener un servicio para consultar los clientes y permitir que tanto el sistema de ventas como el de facturación utilicen esa misma función.
- **Problema a resolver:** Busca facilitar la conexión entre diferentes programas y evitar que cada uno tenga que crear las mismas funciones por separado.
- **Competentes principales y función de cada uno:**

  Proveedor del servicio: es el sistema que ofrece una función.

  Consumidor del servicio: es el programa que utiliza esa función.

  Contrato del servicio: explica qué operaciones ofrece y cómo se deben solicitar.

  Bus de servicios empresariales (ESB): cuando se utiliza, ayuda a dirigir mensajes y conectar sistemas diferentes.

  Sistemas empresariales: son los programas que necesitan intercambiar información.

- **Forma en que se comunican los componentes:**

Los programas solicitan las funciones siguiendo unas reglas acordadas. Por ejemplo, el sistema de ventas puede pedir información de un cliente al servicio de clientes. En algunas empresas, un bus de servicios se encarga de dirigir la solicitud al lugar correcto.

- **Ventajas y Desventajas:**

Ventajas	**Desventajas**

permite aprovechar una misma función en varios sistemas. Por ejemplo, el sistema de ventas y el de facturación pueden utilizar el mismo servicio para consultar los datos de un cliente.	organizar las comunicaciones entre muchos sistemas puede ser complicado, porque cada uno debe respetar las reglas acordadas.
ayuda a conectar programas antiguos con aplicaciones nuevas, aunque hayan sido desarrollados con tecnologías diferentes.	si toda la comunicación depende de un solo bus de servicios y este falla, varias funciones pueden verse afectadas si no existe una alternativa.

- **Un ejemplo real o técnicamente verificable en uso:**

Una empresa que conecta sus sistemas de ventas, inventario y facturación mediante servicios web, por ejemplo, utilizando SOAP para intercambiar información.

- **Un proyecto de software en el que sería apropiada:**

La utilizaría en una institución educativa que necesite conectar los sistemas de matrículas, pagos, certificados y notas para compartir información entre ellos.

- **Una situación en la que no recomendaría utilizarla y la justificación:**

No la utilizaría en una aplicación pequeña que solamente tenga unos pocos módulos y no necesite conectarse con otros sistemas. Crear toda una estructura de servicios e integración podría complicar el proyecto sin aportar una ventaja importante.

- **Fuente técnica consultada y el dato especifico que tuve de ella:**

Microsoft Learn: Azure Architecture Center

Esto explica cómo conectar aplicaciones y compartir información mediante servicios y mecanismos de comunicación. Esto ayuda a entender por qué SOA puede ser útil cuando una empresa necesita unir varios programas.

6. Arquitectura dirigida por eventos (Event-Driven)

- **Definición:** Es una arquitectura en la que algunas partes del programa realizan acciones cuando ocurre algo importante, llamado evento. Por ejemplo, cuando un estudiante termina una evaluación, el sistema puede generar un evento para guardar el resultado y otro proceso puede enviarle una notificación.
- **Problema a resolver:** Busca que diferentes partes del sistema puedan reaccionar a una misma acción sin que la parte que la produce tenga que encargarse directamente de todas las tareas relacionadas.
- **Componentes principales y función de cada uno:**

  Productor de eventos: es la parte que informa que ocurrió algo.

  Canal o intermediario de eventos: recibe los eventos y los distribuye.

  Consumidores: son las partes que reciben los eventos y realizan una acción.

  Almacenamiento de eventos: en algunos sistemas se utiliza para guardar un historial de los eventos ocurridos.

- **Forma en que se comunican los componentes:**

Una parte del sistema envía un mensaje para informar que ocurrió algo y las demás partes interesadas lo reciben. Por ejemplo, cuando un estudiante se inscribe en un curso, un proceso puede actualizar las estadísticas y otro puede enviar un correo de confirmación.

- **Ventajas y desventajas**

Ventajas	Desventajas
permite agregar nuevas funciones que reaccionen a los eventos sin tener que cambiar demasiado el código que los genera. Por ejemplo, podemos añadir un servicio de estadísticas que reciba información sobre las inscripciones.	la información no siempre se actualiza inmediatamente en todas las partes. Por ejemplo, las estadísticas pueden tardar un poco en mostrar una inscripción nueva.
algunas tareas pueden realizarse por separado, sin tener que esperar a que todas terminen al mismo tiempo.	hay que controlar que los mensajes no se pierdan ni se procesen varias veces de una forma que cause problemas.

- **Un ejemplo real o técnicamente verificable de uso:**

Una aplicación que utilice Apache Kafka o Azure Event Grid para enviar información sobre las acciones que ocurren en el sistema.

- Un proyecto de software en el que sería apropiada:

La utilizaría en PlayNoteMusic para que, al terminar una evaluación, el sistema actualice las estadísticas del estudiante y genere una notificación sin tener que realizar todas esas tareas dentro de una sola función.

- **Una situación en la que no recomendaría utilizarla y la justificación:**

No la utilizaría para todas las operaciones de un programa pequeño. Por ejemplo, para comprobar si una contraseña es correcta, una

solicitud directa puede ser más sencilla que crear un sistema de eventos.

- **Una fuente técnica consultada y el dato especifico que obtuve de ella:**

Microsoft Learn: Event-Driven Architecture Style

Esto habla de cómo los productores generan eventos, los canales los distribuyen y los consumidores realizan acciones cuando los reciben. También permite entender que estas partes pueden trabajar de forma separada.

7. Arquitectura serverless (sin servidor)

- **Definición:** La arquitectura serverless permite ejecutar funciones de una aplicación sin tener que administrar directamente los servidores. El proveedor de la nube se encarga de buena parte de esa administración.
- **Problema a resolver:** Busca reducir el trabajo de mantener servidores y permitir que ciertas funciones se ejecuten cuando se necesitan. Por ejemplo, se puede crear una función que envíe un correo cuando un estudiante solicite recuperar su contraseña.
- **Componentes principales y función de cada uno:**

  Disparadores: son los que inician una función, como una solicitud desde una página web.

  Funciones: realizan las tareas programadas.

Servicios de almacenamiento: guardan archivos o información cuando se necesitan.

Proveedor de nube: proporciona la infraestructura donde se ejecutan las funciones.

Monitoreo: ayuda a revisar los errores y el uso del servicio.

- **Forma en que se comunican los componentes:**

Una acción inicia la función y esta realiza el trabajo correspondiente. Por ejemplo, un estudiante solicita recuperar su contraseña, la petición activa una función y esta procesa la solicitud para enviar el correo de recuperación.

- **Ventajas y desventajas:**

Ventajas	Desventajas
Se necesita administrar menos infraestructura: el desarrollador puede concentrarse en programar la función sin tener que configurar y mantener directamente un servidor.	El costo depende del servicio y de su uso: si la aplicación tiene muchísimas ejecuciones, el cobro podría aumentar. Por eso, hay que revisar las condiciones del proveedor.
Puede adaptarse a la cantidad de solicitudes: algunas plataformas aumentan o reducen automáticamente los recursos según el uso.	Algunas funciones pueden tardar en iniciar: después de un tiempo sin utilizarse, ciertas plataformas pueden tardar más en ejecutar la primera solicitud.

- **Un ejemplo real o técnicamente verificable de uso:**

AWS Lambda y Azure Functions son servicios que permiten ejecutar funciones en la nube sin administrar directamente los servidores donde se ejecutan.

- **Un proyecto de software en el que sería apropiada:**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

La utilizaría para enviar correos de recuperación de contraseñas, procesar imágenes de instrumentos o realizar tareas que no necesiten ejecutarse constantemente.

- **Una situación en la que no recomendaría utilizarla y la justificación:**

No la elegiría sin comparar otras opciones para un sistema que deba ejecutar procesos de forma continua y estable. En ese caso, podría ser más práctico o económico utilizar un servidor o contenedores, dependiendo de los costos y las necesidades.

- **Una fuente técnica consultada y el dato especifico que tuvo de ella:**

Microsoft Learn, Estilos de arquitectura

Esto permite comparar diferentes formas de utilizar los servicios de nube y organizar los componentes de una aplicación. La elección debe tener en cuenta las necesidades del sistema y la forma en que se van a ejecutar sus tareas.

8. Arquitectura microkernel

- **Definición:** La arquitectura microkernel consiste en tener una parte principal del programa que contiene las funciones más importantes y permitir que se agreguen otras funciones mediante complementos, también conocidos como plugins.
- **Problema a resolver:** Busca que una aplicación pueda tener nuevas funciones sin necesidad de modificar todo el programa cada vez que se quiera agregar algo.
- **Componentes principales y función de cada uno:**

  Núcleo: es la parte principal que permite que el programa funcione.

  Complementos: agregan nuevas funciones al programa.

  Interfaces: establecen las reglas que deben cumplir los complementos para funcionar con el núcleo.

  Administrador de complementos: si el sistema lo incluye, se encarga de cargar y administrar los complementos instalados.

- **Forma en que se comunican los componentes:**

Los complementos utilizan las interfaces que ofrece el núcleo para poder trabajar con el programa. Por ejemplo, un editor de código puede tener un núcleo básico y permitir instalar complementos para trabajar con nuevos lenguajes de programación.

- **Ventajas y desventajas:**

Ventajas	Desventajas
Se pueden agregar funciones cuando se necesiten: no hace falta incluir todas las herramientas desde el	Hay que definir bien cómo funcionan los complementos: si el núcleo cambia una interfaz importante, algunos

principio. El usuario puede instalar solamente las que vaya a utilizar.	complementos podrían dejar de funcionar.
El programa principal puede mantenerse más sencillo: las funciones adicionales se desarrollan como complementos separados.	Un complemento puede causar problemas: si está mal programado o no es compatible, podría provocar errores en la aplicación.

- **Un ejemplo real o técnicamente verificable de uso:**

Eclipse IDE, un entorno de desarrollo que permite agregar herramientas y otras funciones mediante complementos.

- **Un proyecto de software en el que sería apropiado:**

La utilizaría en un editor de partituras musicales que permita instalar complementos para aprender guitarra, piano y otros instrumentos, sin incluir todas las funciones desde el principio.

- **Una situación en el que no reconocería utilizarla y la justificación:**

No la recomendaría para un programa pequeño que solo tiene dos o tres funciones y no necesita agregar nuevas herramientas. Preparar un sistema de complementos podría complicar un proyecto que realmente es sencillo.

- **Una fuente técnica consultada y el dato especifico que tuvo de ella:**

Eclipse IDE

Eclipse es un entorno de desarrollo que permite ampliar sus funciones mediante complementos. Este ejemplo muestra cómo un programa puede ofrecer herramientas adicionales sin tener que crear un programa completamente nuevo para cada función.

9. Arquitectura hexagonal

- **Definición**: Esta arquitectura busca separar las reglas principales de un programa de las tecnologías que utiliza para funcionar. Por ejemplo, las reglas de inscripción deberían estar separadas de la página web y de la base de datos.
- **Problema a resolver:** Busca que los cambios en una tecnología no obliguen a modificar todas las reglas del programa. Por ejemplo, si se cambia MySQL por otra base de datos, la lógica de las inscripciones podría seguir funcionando si se mantiene la misma interfaz de comunicación.
- **Componentes principales y función de cada uno:**

  Núcleo del programa: contiene las reglas principales del negocio.

  Puertos: son las interfaces que indican cómo puede comunicarse el núcleo con otras partes.

  Adaptadores de entrada: permiten que lleguen solicitudes desde una página web, una API o una prueba.

  Adaptadores de salida: conectan el núcleo con elementos externos, como una base de datos o un servicio de correo.

- **Forma en que se comunican los componentes.**

Las partes externas se conectan con el núcleo mediante los puertos y adaptadores. Por ejemplo, cuando un estudiante se inscribe en un curso, la página envía los datos al núcleo del programa. Esta comprueba las reglas de inscripción y utiliza un adaptador para guardar los datos en la base de datos.

- Ventajas y Desventajas:

Ventajas	**Desventajas**

Facilita hacer pruebas: se pueden probar las reglas de inscripción sin tener que utilizar siempre una página web o una base de datos real.	Se crean más archivos e interfaces: esto requiere más trabajo al principio que una estructura sencilla.
Permite cambiar algunas tecnologías con menos trabajo: si se mantienen las interfaces, se puede cambiar el adaptador que conecta con la base de datos sin tener que rehacer todas las reglas.	Puede ser difícil de entender al comenzar: si todavía no se tiene mucha experiencia programando, puede costar seguir el recorrido de una operación entre el núcleo y los adaptadores.

- **Un ejemplo real o técnicamente verificable de uso:**

Una aplicación desarrollada con Java que separa las reglas para registrar estudiantes de los componentes encargados de recibir solicitudes web y guardar información.

- **Un proyecto de software en el que sería apropiado:**

La utilizaría en PlayNoteMusic para separar las reglas de inscripción y evaluación de las pantallas hechas con React y de la base de datos MySQL.

- **Una situación en el que no reconocería utilizarla y la justificación:**

No la utilizaría con una estructura muy elaborada para una página pequeña que solo muestra información estática. En un caso así, sería demasiado trabajo crear todos esos elementos si no se necesitan cambios frecuentes en las tecnologías.

- **Una fuente técnica consultada y el dato especifico que tuvo de ella:**

Alistair Cockburn, Hexagonal Architecture

El autor explica que la aplicación puede conectarse con usuarios, bases de datos y otros sistemas por medio de puertos y adaptadores. Esto ayuda a realizar pruebas sin depender directamente de las tecnologías externas.

10. Arquitectura peer-to-peer (P2P)

- **Definición:** La arquitectura P2P permite que varios computadores se conecten entre sí para compartir información. Cada equipo puede recibir información de otros y también compartir la que tiene.
- **Problema a resolver:** Busca compartir archivos y recursos sin depender completamente de un solo servidor que tenga que enviar toda la información.
- **Componentes principales y función de cada uno:**

  Pares o nodos: son los computadores que participan en la red.

  Mecanismo para encontrar otros equipos: ayuda a localizar a los participantes con los que se puede compartir información.

  Protocolo de comunicación: establece las reglas para enviar y recibir datos.

  Mecanismos de seguridad: ayudan a comprobar que la información recibida sea correcta y que los participantes tengan los permisos necesarios.

- **Forma en que se comunican los componentes.**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Los equipos se conectan y comparten información siguiendo las reglas del protocolo utilizado. Por ejemplo, cuando una persona descarga un archivo por medio de una red P2P, puede recibir diferentes partes de ese archivo desde varios equipos que también lo están compartiendo.

- **Ventajas y desventajas:**

Ventajas	Desventajas
La información se puede repartir entre varios equipos: no es necesario que un solo computador envíe todo el archivo a todas las personas.	Los equipos pueden desconectarse: si algunos participantes se van de la red, encontrar la información puede volverse más difícil.
Puede reducir la dependencia de un servidor central: los participantes ayudan a compartir los recursos disponibles en la red.	Hay que tener cuidado con la seguridad: no se debe confiar automáticamente en cualquier archivo recibido de otro participante, porque podría estar dañado o no ser seguro.

- **Un ejemplo real o técnicamente verificable de uso:**

BitTorrent, un protocolo utilizado para compartir archivos entre diferentes equipos conectados a una red P2P.

- **Un proyecto de software en el que sería apropiado:**

La utilizaría en un proyecto para distribuir archivos grandes de recursos musicales, como grabaciones de práctica o materiales de aprendizaje, entre varios participantes.

- **Una situación en el que no reconocería utilizarla y la justificación:**

No la utilizaría como única forma de guardar y compartir las calificaciones privadas o las contraseñas de los estudiantes. Ese tipo

de información necesita controles de seguridad y permisos bien definidos.

- **Una fuente técnica consultada y el dato especifico que tuvo de ella:**

BitTorrent, Especificación del protocolo

La documentación explica que los participantes pueden intercambiar datos en ambas direcciones y compartir partes de archivos con otros participantes de la red.

11. Arquitectura basada en componentes

- **Definición:** Esta arquitectura consiste en construir un programa utilizando partes más pequeñas, donde cada una tiene una función específica. Esas partes se pueden reutilizar en diferentes lugares de la aplicación. Por ejemplo, una tarjeta diseñada para mostrar un curso también podría utilizarse para mostrar otros cursos cambiando su título, imagen y descripción.
- **Problema a resolver:** Busca evitar que se repita el mismo código muchas veces y que todo el programa quede concentrado en un único archivo demasiado grande.
- **Componentes principales y función de cada uno:**

  Componentes: son las partes que realizan una función, como mostrar tarjetas, formularios o tablas.

Interfaces: establecen cómo se puede utilizar cada componente y qué información necesita.

Mecanismo para combinar componentes: permite unir varias partes para construir una pantalla.

Aplicación principal: reúne los componentes y permite que el sistema funcione como un conjunto.

- **Forma en que se comunican los componentes.**

Los componentes intercambian información según la tecnología utilizada. Por ejemplo, en React se pueden enviar datos de un componente a otro mediante props. Si tenemos un componente llamado Card, podemos utilizarlo varias veces y cambiar los datos que muestra sin tener que escribir toda la tarjeta nuevamente.

- **Ventajas y Desventajas:**

Ventajas	Desventajas
Se puede reutilizar el código: si se necesita mostrar diez tarjetas de cursos, se puede utilizar el mismo componente cambiando la información.	Un componente puede quedar demasiado grande: si se le agregan muchas responsabilidades, se vuelve difícil de entender y modificar.
Es más fácil organizar las pantallas: cada componente tiene una tarea y, cuando hay un problema, se puede revisar la parte correspondiente.	Los componentes pueden depender demasiado unos de otros: si se cambia la forma de enviar información, podrían aparecer errores en otros componentes que utilizan esa misma parte.

- **Un ejemplo real o técnicamente verificable de uso:**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

React, una tecnología que permite construir interfaces utilizando componentes reutilizables para crear botones, tarjetas, formularios y otras partes de una página web.

- **Un proyecto de software en el que sería apropiado:**

La utilizaría en PlayNoteMusic para crear tarjetas de cursos, formularios de inscripción, tablas de evaluaciones y otros elementos que se necesitan mostrar en varias pantallas.

- **Una situación en el que no reconocería utilizarla y la justificación:**

No crearía muchos componentes para una página muy sencilla que solo tenga un título y un párrafo. En ese caso, separar cada elemento podría hacer que el proyecto tenga más archivos de los necesarios.

- **Una fuente técnica consultada y el dato especifico que tuvo de ella:**

React, Your First Component

La documentación explica que los componentes son partes que se pueden utilizar para construir interfaces más grandes. Esto permite organizar una página y reutilizar elementos en diferentes lugares.

12. Arquitectura broker/pipeline

A. Arquitectura broker

- **Definición:** Consiste en utilizar un intermediario que recibe mensajes o solicitudes y se encarga de enviarlos al lugar correspondiente. Por ejemplo, si varios servicios necesitan recibir mensajes de una aplicación, un broker puede ayudar a dirigir cada mensaje al servicio adecuado.
- **Problema a resolver:** Busca facilitar la comunicación entre diferentes partes del programa para que no tengan que conectarse directamente unas con otras.
- **Componentes principales y función de cada uno:**

  Emisor: es la parte que envía el mensaje.

  Broker: recibe el mensaje y decide a dónde enviarlo.

  Receptor: es la parte que recibe el mensaje y realiza la tarea.

  Colas de mensajes: pueden guardar los mensajes hasta que el receptor esté preparado para procesarlos.

- **Forma en que se comunican los componentes.**

El emisor manda el mensaje al broker y este lo envía al receptor correspondiente. Por ejemplo, cuando un estudiante solicita recuperar su contraseña, una aplicación puede enviar una solicitud que se dirige al servicio encargado de enviar los correos.

- **Ventajas y Desventajas:**

Ventajas	Desventajas
Facilita agregar nuevos servicios: el emisor no necesita conocer todos los	El broker también puede fallar: si se depende de un único intermediario y

detalles de cada receptor, porque el broker ayuda a dirigir los mensajes.	este deja de funcionar, la comunicación puede verse afectada.
Permite organizar las solicitudes: algunos brokers pueden guardar mensajes en colas para que se procesen cuando el servicio receptor esté disponible.	Requiere configuración adicional: hay que definir cómo se envían y reciben los mensajes y qué debe suceder cuando ocurre un error.

- **Un ejemplo real o técnicamente verificable de uso:**

RabbitMQ, una herramienta que permite recibir mensajes y dirigirlos hacia las colas correspondientes para que otras aplicaciones los procesen.

- **Un proyecto de software en el que sería apropiado:**

La utilizaría en PlayNoteMusic si se necesita enviar mensajes a varios servicios, por ejemplo, para notificaciones, correos electrónicos y procesamiento de actividades.

- **Una situación en el que no reconocería utilizarla y la justificación:**

No la utilizaría para comunicar solamente dos partes sencillas de un programa si pueden trabajar directamente sin problemas. En ese caso, añadir un broker podría complicar innecesariamente el proyecto.

- **Una fuente técnica consultada y el dato especifico que tuvo de ella:**

RabbitMQ, Conceptos de AMQP

La documentación explica que los intercambios reciben los mensajes y los dirigen hacia una o varias colas según las reglas configuradas.

Esto permite que el emisor no tenga que enviar cada mensaje directamente a su receptor final.

B. Arquitectura pipeline

- **Definición:** La arquitectura pipeline consiste en organizar un proceso en varios pasos. Cada paso realiza una tarea y después entrega el resultado al siguiente. Por ejemplo, para calificar un examen, primero se leen las respuestas, después se revisan, luego se calcula la nota y finalmente se guarda el resultado.
- **Problema a resolver:** Busca organizar los procesos que necesitan realizar varias operaciones seguidas, para que el código no tenga que hacer todo dentro de una sola función.
- **Componentes principales y función de cada uno:**

  Entrada: recibe la información que se va a procesar.

  Etapas o filtros: son las partes que realizan cada tarea.

  Conexiones o tuberías: permiten pasar la información de una etapa a la siguiente.

  Salida: recibe o guarda el resultado final del proceso.

- **Forma en que se comunican los componentes.**

La información va pasando de una etapa a otra. Cada parte recibe los datos, realiza su tarea y entrega el resultado a la siguiente. Por ejemplo, un sistema puede recibir las respuestas de un estudiante,

revisar que estén completas, calcular la calificación y guardar el resultado en la base de datos.

- **Ventajas y Desventajas:**

Ventajas	Desventajas
Es más fácil encontrar errores: si un archivo no pasa la validación, se puede revisar esa etapa sin tener que investigar todo el proceso.	Puede tardar más en completar el proceso: la información tiene que pasar por varias etapas y cada una necesita realizar su trabajo.
Se pueden reutilizar las etapas: una misma función para validar datos podría utilizarse en diferentes procesos del sistema.	Un error puede afectar las etapas siguientes: si una parte entrega información incorrecta o no puede continuar, el proceso podría quedar incompleto.

- **Un ejemplo real o técnicamente verificable de uso:**

Los comandos de Unix que se conectan mediante tuberías. Por ejemplo, la salida de un comando puede utilizarse como entrada de otro para procesar información paso a paso.

- **Un proyecto de software en el que sería apropiado:**

La utilizaría para procesar los resultados de las evaluaciones de PlayNoteMusic: recibir las respuestas, comprobarlas, calcular la nota y guardar los resultados.

- **Una situación en el que no reconocería utilizarla y la justificación:**

No la utilizaría como estructura principal de una aplicación cuando sus funciones son independientes y no necesitan realizarse en un orden específico. En ese caso, organizar todo como un pipeline podría complicar el desarrollo sin aportar una ventaja clara.

- **Una fuente técnica consultada y el dato especifico que tuvo de ella:**

Microsoft Learn, Patrón Pipes and Filters

Esto explica que el proceso se puede dividir en filtros independientes, donde cada uno realiza una tarea y entrega su resultado al siguiente mediante tuberías.

4. Fase II. Análisis comparativo

Seleccione cuatro arquitecturas de las doce investigadas y responda:

1. ¿Qué diferencia fundamental existe entre las cuatro?
2. ¿Cuáles permiten separar mejor las responsabilidades? Explique por qué.
3. ¿Cuáles facilitan el despliegue independiente de partes del sistema?
4. ¿Qué arquitecturas requieren considerar especialmente la comunicación entre componentes?

5. ¿Cuál podría aumentar la complejidad de operación de un proyecto pequeño?
6. ¿Es posible combinar varios estilos de arquitectura en un mismo sistema? Justifique con un ejemplo.
7. ¿Qué arquitectura elegiría para una aplicación con pocos usuarios y presupuesto limitado? ¿Cambiaría su elección si la aplicación tuviera millones de usuarios? Explique qué otros factores evaluaría.

Las respuestas deben presentar argumentos técnicos, no solamente nombrar arquitecturas.

Solución:

1. La diferencia está en cómo se organiza el programa. La arquitectura monolítica tiene las funciones en una misma aplicación. La arquitectura en capas organiza el código por sus funciones, como las pantallas, las reglas y la base de datos. Los microservicios separan las funciones en varios servicios y la arquitectura dirigida por eventos permite que unas partes reaccionen cuando ocurre algo en el sistema.

2. Las arquitecturas en capas y microservicios permiten separar mejor las responsabilidades. En las capas, cada parte tiene una función, por ejemplo, una se encarga de mostrar la información y otra de guardarla. En los microservicios, cada servicio cumple una tarea diferente, como administrar los usuarios o los cursos. Esto ayuda a hacer cambios sin tener que modificar todo el programa.

3. Los microservicios facilitan más el despliegue independiente, porque se puede actualizar un servicio sin publicar nuevamente toda la aplicación, siempre que no afecte a los demás. Por ejemplo, se puede mejorar el servicio de cursos sin cambiar el de usuarios. En cambio, en una aplicación monolítica normalmente se actualiza el programa completo.

4. Principalmente, los microservicios y la arquitectura dirigida por eventos. En los microservicios, las diferentes partes necesitan comunicarse para realizar algunas tareas. En los eventos, los mensajes deben llegar a las partes que necesitan reaccionar. Si la comunicación falla, algunas funciones podrían no completarse correctamente.

5. Los microservicios, porque hay que administrar varios servicios y revisar que todos funcionen y se comuniquen bien. Por ejemplo, para una aplicación pequeña que solamente registra usuarios y muestra cursos, sería más trabajo del necesario. En ese caso, sería mejor empezar con una aplicación sencilla.

6. Sí, se pueden combinar. Por ejemplo, en PlayNoteMusic se podría utilizar una arquitectura monolítica organizada en capas para tener el código ordenado. Después, se podrían agregar eventos para enviar notificaciones cuando un estudiante termine una evaluación. Así se aprovechan diferentes estilos según lo que necesite el proyecto.

7. Para una aplicación con pocos usuarios elegiría la arquitectura monolítica en capas, porque es más sencilla de desarrollar y permite tener el código organizado sin gastar tantos recursos.

   Si tuviera millones de usuarios, utilizaría microservicios para separar las funciones que más se utilizan y poder ampliar esas partes cuando sea necesario.

   No significa que por tener muchos usuarios haya que cambiar toda la arquitectura de inmediato, sino que primero se debe revisar qué problemas tiene y cuál sería la mejor manera de solucionarlos.

5. Fase III. Caso práctico individual

A. Proponga una arquitectura inicial

1. Arquitectura que utilizaría

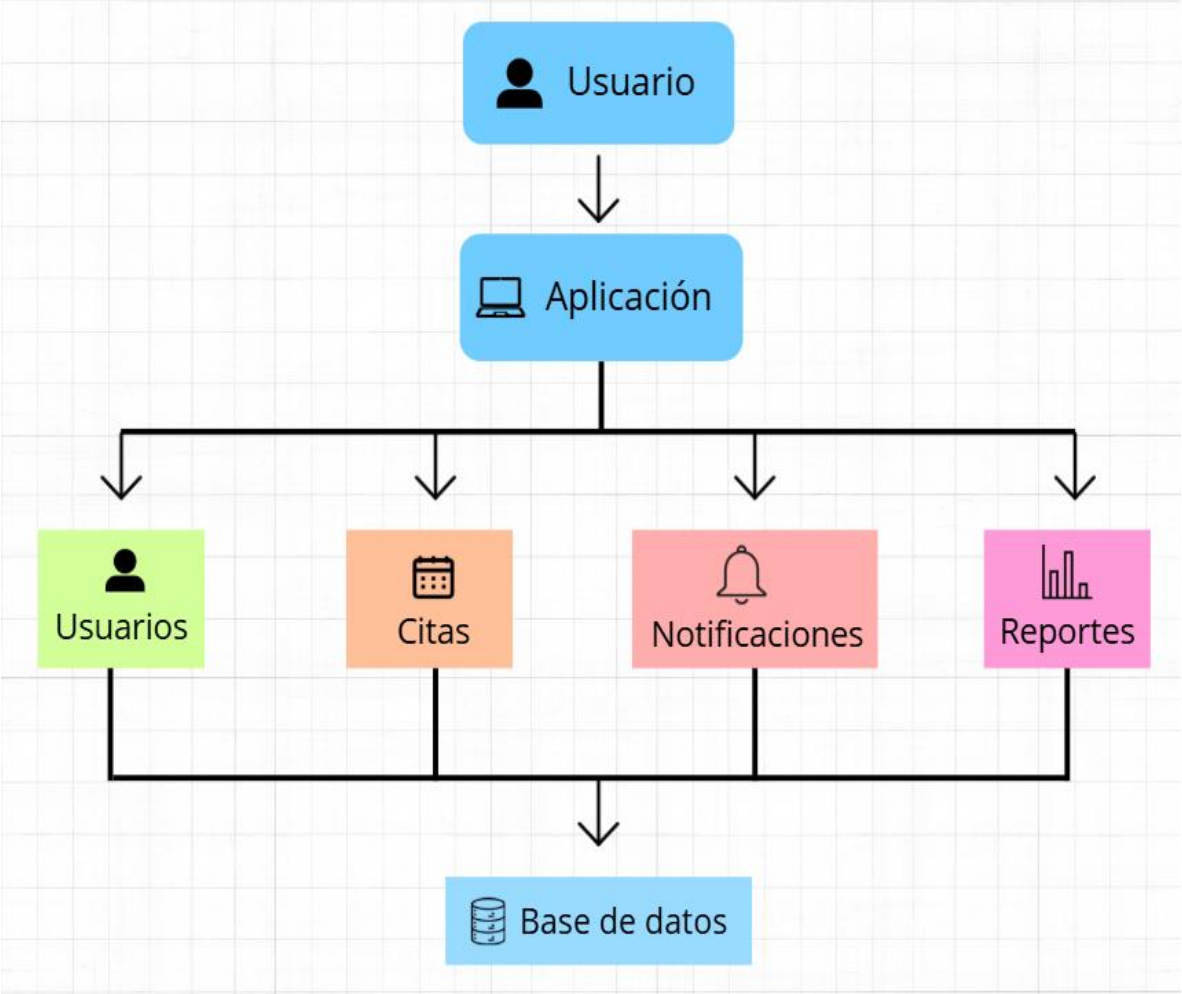
Yo utilizaría una arquitectura monolítica organizada en capas combinada con eventos para enviar las notificaciones automáticamente.

La escogería porque el sistema va a comenzar con pocos usuarios y sería más sencillo desarrollar y mantener una sola aplicación. Al mismo tiempo, organizaría el código por funciones para que sea más fácil hacer cambios si el sistema crece.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

2. Diagrama con los componentes principales.

Diagrama Inicial (Borrador)



3. Responsabilidades de cada componente.

Interfaz web: permite que los usuarios registren citas y consulten la información.

Usuarios: administra los datos de las personas registradas.

Citas: permite crear, consultar y modificar las citas.

Reportes: muestra información sobre las citas, como cuántas se han realizado.

Base de datos: guarda la información de los usuarios y las citas.

Cola de mensajes: recibe los avisos de las citas registradas para que se procesen después.

Notificaciones: utiliza esos avisos para enviar la confirmación por correo o mensaje de texto.

4. Como viajarían los datos.

Primero, el usuario llena el formulario para registrar una cita. La aplicación revisa los datos y comprueba que la cita se pueda registrar.

Después, la cita se guarda en la base de datos y el sistema genera un evento indicando que se registró una nueva cita. Ese aviso llega a la cola de mensajes y el servicio de notificaciones lo procesa para enviar la confirmación al usuario.

De esta manera, el usuario no tiene que esperar a que termine todo el proceso de envío para que se registre su cita. Además, el módulo de reportes puede consultar la información guardada para generar los informes necesarios.

B. Analice dos alternativas.

1 alternativa – Arquitectura de microservicios

Esta arquitectura permitiría separar usuarios, citas, notificaciones y reportes en servicios independientes.

Ventaja: si el módulo de reportes necesita más capacidad, se podría ampliar sin tener que ampliar todo el sistema.

Dificultad: habría que administrar varios servicios y sus comunicaciones. Para un proyecto que apenas comienza, esto podría aumentar el trabajo.

2 alternativa – Arquitectura serverless

Esta opción permitiría utilizar funciones en la nube para enviar notificaciones y realizar ciertas tareas cuando se necesiten.

Ventaja: no sería necesario administrar directamente los servidores donde se ejecutan esas funciones, lo que podría facilitar el envío automático de notificaciones.

Dificultad: se dependería de los servicios de un proveedor de nube y sería necesario revisar los costos de acuerdo con el uso. También habría que aprender a configurar esos servicios.

C. Identifique riesgos.

1. Que la notificación no llegue. Puede ocurrir un error al enviar el correo o el mensaje. Para evitarlo, guardaría los mensajes pendientes y programaría nuevos intentos de envío cuando ocurra un fallo.

2. Que se registren citas duplicadas. Un usuario podría enviar varias veces el formulario por error. Para prevenirlo, validaría los datos antes de guardar la cita y controlaría las solicitudes repetidas.

3. Que el sistema se vuelva lento cuando aumenten los usuarios. Si muchas personas consultan citas o generan reportes al mismo tiempo, la aplicación podría tardar en responder. Para solucionarlo, revisaría qué parte está causando la demora y mejoraría esa parte antes de considerar una separación en microservicios.

D. Justifique su decisión.

Yo escogería la arquitectura en capas porque es más fácil para empezar y permite tener el código organizado. Si el sistema crece y los reportes se vuelven más pesados, buscaría una forma de separarlos para que funcionen mejor. También cambiaría la arquitectura si la aplicación empieza a ponerse lenta o tiene muchos más usuarios.

Empezaría con algo sencillo y a medida que el sistema crezca haría los cambios que sean necesarios.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

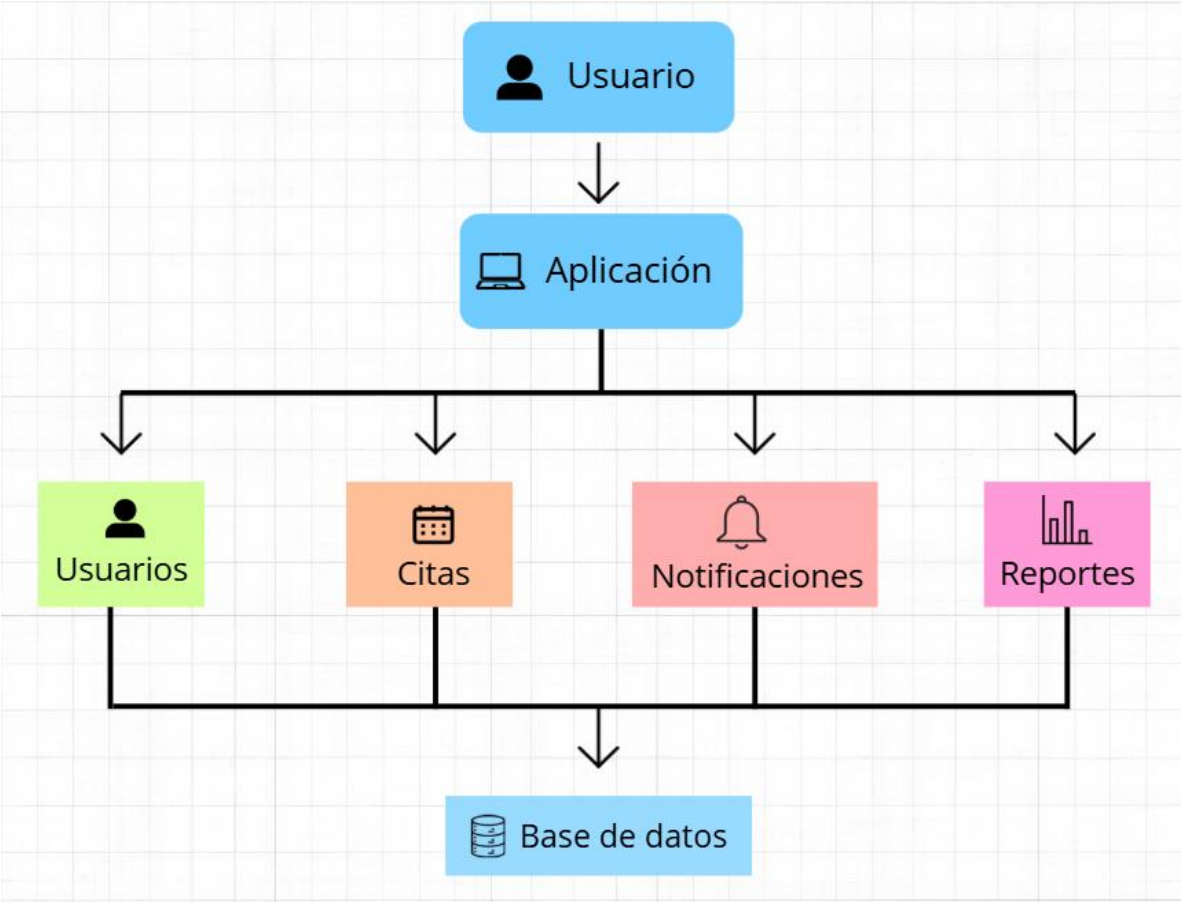
6. Fase IV. Evidencias del proceso

1. **Registro de fuentes consultadas, con enlaces y fecha de consulta.**

Fuente consultada	¿Para qué me sirvió?	Fecha de consulta
Microsoft Learn – Estilos de arquitectura	Para conocer diferentes tipos de arquitectura y sus características.	13/10/2026
Microsoft Learn – Arquitectura en capas	Para entender cómo se puede organizar una aplicación por partes.	13/10/2026
Microsoft Learn – Arquitectura dirigida por eventos	Para conocer cómo se pueden procesar los avisos cuando ocurre una acción en el sistema.	13/10/2026

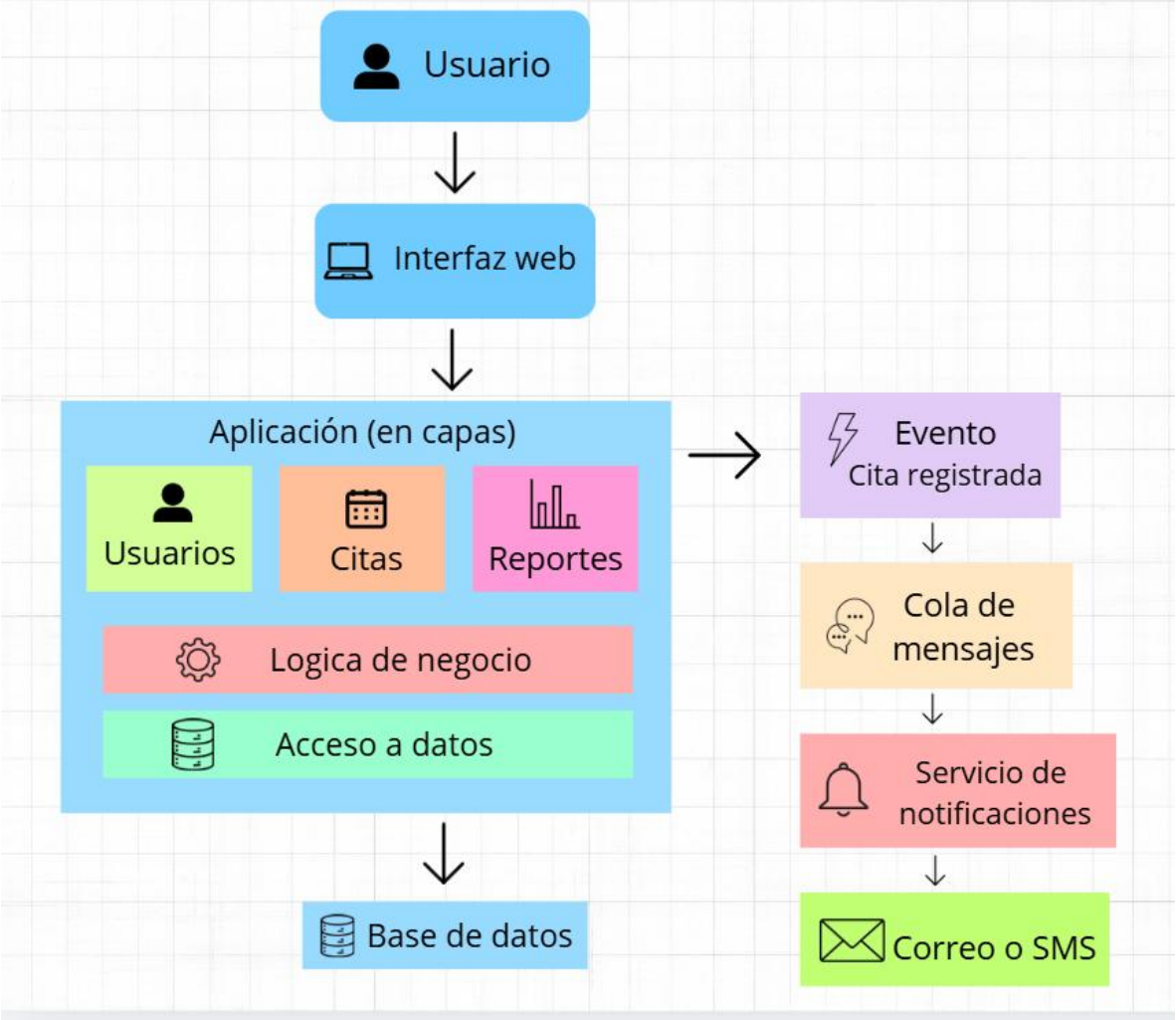
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

2. Primer esquema o borrador del diagrama.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

3. Versión final del diagrama con las correcciones realizadas.



4. Tabla de decisiones técnicas: decisión tomada, alternativa descartada y justificación.

Decisión tomada	Alternativa descartada	Justificación

Elegir la arquitectura en capas.	Microservicios.	Elegí las capas porque permiten organizar las funciones y empezar con una aplicación más sencilla.
Manejar las notificaciones como una función separada dentro del sistema.	Mezclar las notificaciones con el registro de las citas.	Así es más fácil organizar los avisos y hacer cambios en esa parte cuando sea necesario.

5. Breve bitácora de las dificultades encontradas y cómo las resolvió.

Durante el trabajo tuve dificultad para elegir la arquitectura más adecuada, porque existen varios tipos y no sabía cuál se ajustaba mejor al sistema de citas.

Para resolverlo, revisé sus características y tuve en cuenta que el sistema comenzará con pocos usuarios, pero podría crecer más adelante. Por eso elegí la arquitectura en capas, ya que permite organizar mejor las funciones.

También tuve dificultad para organizar el diagrama. Lo solucioné separando las partes principales, como usuarios, citas, notificaciones, reportes y base de datos, para que fuera más fácil entender cómo funciona el sistema.